- Access modifier is public.
- Return type is String.
- The method belongs to StudentService.
- Method name is getStudent.
- It accepts one String parameter.

The access modifier can be omitted:

```
@Before("execution(String com.example.aopdemo.service.StudentService.getStudent(String))")
```

If a method accepts a Student object and returns a Student object, specify the fully qualified type name in the expression:

```
@Before("execution(com.example.aopdemo.dto.Student com.example.aopdemo.service.StudentService.createStudent(com.example.aop
```

4. Why Wildcards Are Needed

Exact Pointcut expressions can become lengthy. If a service contains many methods, creating a separate Pointcut for every method becomes repetitive. Wildcards allow patterns to match multiple methods.

Wildcard	Meaning
*	Matches one item in the relevant position.
..	Matches zero or more parameters or package levels, depending on where it is used.

5. Using * in Pointcut Expressions

5.1 Any return type

```
execution(* com.example.aopdemo.service.StudentService.getStudent(String))
```

The first * means any return type, such as String, Integer, Student, or void.

5.2 Any single parameter type

```
execution(* com.example.aopdemo.service.StudentService.getStudent(*))
```

The parameter wildcard matches exactly one parameter of any type. It does not match two parameters.

5.3 Multiple parameters

```
execution(* com.example.aopdemo.service.StudentService.getStudent(*,*))
```

This matches two parameters of any types. To require the second parameter to be Integer:

```
execution(* com.example.aopdemo.service.StudentService.getStudent(*,Integer))
```

5. Using * in Pointcut Expressions (continued)

5.4 Any method name

```
execution(* com.example.aopdemo.service.StudentService.*(*))
```

This matches any method name in StudentService that accepts one parameter, regardless of return type.

5.5 Method-name patterns

Pattern	Meaning
get*	Method names beginning with get
*Student	Method names ending with Student
*	Any method name

```
execution(* com.example.aopdemo.service.StudentService.get*(*))
```

5.6 Matching classes and methods inside a package

```
execution(* com.example.aopdemo.service.*.*(*))
```

This matches any method with one parameter in any class directly inside the service package. The package-position * does not recursively match subpackages.

6. Using .. in Pointcut Expressions

6.1 Any number of parameters

```
execution(* com.example.aopdemo.service.StudentService.*(..))
```

The .. parameter pattern matches zero, one, two, or more parameters of any types.

6.2 Difference between * and ..

Expression	Meaning
getStudent(*)	Exactly one parameter of any type
getStudent(..)	Zero or more parameters
getStudent(String)	Exactly one String parameter
getStudent(String,Integer)	Exactly two specified parameter types

6.3 Matching subpackages

```
service package
    |
    v
subpackages
    |
    v
classes
    |
    v
methods
```

```
execution(* com.example.aopdemo.service..*.*(..))
```

This matches methods in classes under the service package, including nested subpackages.

6.4 Matching methods throughout the application

```
execution(* *(..))
```

This is extremely broad. The lecture demonstrates that Spring may attempt to proxy framework methods that cannot be proxied in the required way, causing application startup errors. Restrict the expression to your own application package:

```
execution(* com.example.aopdemo..*.*(..))
```

Key lesson: Avoid unnecessarily broad Pointcuts. Scope them to application packages or classes where the Advice is intended to apply.

7. execution() vs within()

Designator	Matching level	Identifies
execution()	Method level	Specific method executions
within()	Class level	Methods belonging to a class or package pattern

Using execution():

```
@Before("execution(* com.example.aopdemo.service.StudentService.*(..))")
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

Using within():

```
@Before("within(com.example.aopdemo.service.StudentService)")
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

Matching a package and its subpackages:

```
@Before("within(com.example.aopdemo.service..*)")
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

Memory trick: execution() describes the method signature; within() describes the class or package containing the method.

8. @annotation Designator

@annotation matches method executions where the method has a particular annotation.

```
@TimeStamp
public Student createStudent(Student student) {
    System.out.println("Student Saved");
    return student;
}
```

```
@Before("@annotation(jdk.jfr.Timestamp)")
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

The expression uses the annotation's fully qualified class name. The lecture suggests custom annotations such as:

Annotation	Possible purpose
@Authenticate	Execute authentication logic
@Audit	Perform audit logging
@EvaluateTime	Measure method execution time

Example: execution-time measurement

```
@Around("@annotation(com.example.annotation.EvaluateTime)")
public Object evaluateTime(ProceedingJoinPoint joinPoint)
        throws Throwable {

    long startTime = System.currentTimeMillis();
    Object result = joinPoint.proceed();
    long endTime = System.currentTimeMillis();

    System.out.println("Execution time: "
            + (endTime - startTime));

    return result;
}
```

Methods marked with @EvaluateTime can use this Advice. This keeps infrastructure logic separate from business logic.

9. bean() Designator

The bean() designator matches Spring beans by their bean names.

```
@Service
public class StudentService {
}
```

By default, the bean name is studentService. Target it as follows:

```
@Before("bean(studentService)")
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

Important: bean() is based on the Spring bean name, not the class name.

10. Combining Pointcut Expressions

Pointcuts can be combined using logical operators:

Operator	Meaning
&&	AND — both conditions must match
||	OR — at least one condition must match
!	NOT — excludes a matching condition

10.1 OR (||)

```
@Before(
    "within(com.example.aopdemo.service.StudentService) || " +
    "within(com.example.aopdemo.controller.StudentController)"
)
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

If either condition matches, the Advice executes.

10.2 AND (&&)

```
@Before(
    "within(com.example.aopdemo.service..*) && " +
    "execution(public * *(..))"
)
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

Both conditions must match: the method belongs to the service package or its subpackages, and it is public.

10.3 NOT (!)

```
@Before(
    "within(com.example.aopdemo..*) && " +
    "!within(com.example.aopdemo.controller.StudentController)"
)
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

This expression excludes methods in StudentController.

11. Named Pointcuts

When the same Pointcut expression is used in multiple Advice methods, copying it repeatedly makes the code difficult to maintain. Named Pointcuts allow the expression to be declared once and reused.

Step 1: Declare a named Pointcut

```
@Pointcut(
    "within(com.example.aopdemo.service..*) && " +
    "execution(public * *(..))"
)
public void logPublicServiceMethods() {
}
```

- Use @Pointcut to declare the expression.
- Give the Pointcut method a meaningful name.
- Keep its body empty.
- The method name represents the Pointcut expression.

Step 2: Reuse the named Pointcut

```
@Before("logPublicServiceMethods()")
public void logBeforeMethod() {
    System.out.println("Before Method");
}

@After("logPublicServiceMethods()")
public void logAfterMethod() {
    System.out.println("After Method");
}
```

An Around Advice can reuse it too:

```
@Around("logPublicServiceMethods()")
public Object logAroundMethod(ProceedingJoinPoint joinPoint)
        throws Throwable {
    System.out.println("Before");
    Object result = joinPoint.proceed();
    System.out.println("After");
    return result;
}
```

12. ApplicationPointcuts Class — Industry Practice

The lecture recommends keeping commonly reused Pointcuts in a separate class.

```
@Aspect
@Component
public class ApplicationPointcuts {

    @Pointcut(
        "within(com.example.aopdemo.controller..*)"
    )
    public void controllerLayer() {
    }

    @Pointcut(
        "within(com.example.aopdemo.service..*)"
    )
    public void serviceLayer() {
    }
}
```

Other Aspects can reuse these definitions:

```
@Before("com.example.aopdemo.aspect.ApplicationPointcuts.serviceLayer()")
public void logServiceMethod() {
    System.out.println("Service method intercepted");
}
```

Benefits:

- Avoids duplicate expressions.
- Keeps Pointcut definitions centralized.
- Makes Aspects easier to read.
- Makes future changes easier.

13. this and target Designators

These designators are important when understanding Spring AOP proxies.

Designator	Meaning
this	Matches the proxy object based on its type.
target	Matches the actual target object based on its type.

```
@Before("this(com.example.aopdemo.service.StudentService)")
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

```
@Before("target(com.example.aopdemo.service.StudentService)")
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

Proxy and target relationship:

```
Controller
    |
    v
Proxy
    |
    v
Target Object
    |
    v
Actual Method Execution
```

The proxy intercepts calls and delegates to the target object.

14. @within and @target

Designator	Meaning
@within	Matches types whose class-level annotation is specified.
@target	Matches target objects whose type has the specified annotation.

```
@Before("@within(org.springframework.stereotype.Service)")
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

```
@Before("@target(org.springframework.stereotype.Service)")
public void logBeforeMethod() {
    System.out.println("Method Intercepted");
}
```

These are class-level annotation matching expressions, unlike @annotation, which matches methods carrying an annotation.

15. args and @args

Designator	Meaning
args	Matches method arguments based on their runtime types.
@args	Matches method arguments based on annotations present on their runtime types.

```
@Before("args(com.example.aopdemo.dto.Student)")
public void logBeforeMethod() {
```

```
    System.out.println("Student argument detected");
}
```

The lecture introduces these designators but does not provide complete worked examples for each one.

16. CGLIB Proxy vs JDK Dynamic Proxy

Spring AOP commonly uses two proxy mechanisms:

CGLIB Proxy	JDK Dynamic Proxy
Creates a subclass of the target class.	Creates a proxy implementing an interface.
Can proxy classes without an interface, subject to class/method restrictions.	Requires an interface.
Proxy is based on the target class.	Proxy is based on the interface.

CGLIB proxy flow

```
StudentController
        |
        v
Student Proxy
        |
        v
StudentService
        |
        v
Actual Method
```

The proxy extends the target class, so it can be compatible with the target class type.

JDK Dynamic Proxy flow

```
StudentController
        |
        v
Proxy
        |
        v
CommonService Interface
        |
        v
StudentService Target
        |
        v
Actual Method
```

The proxy implements the common interface and delegates to the target implementation.

Why use an interface?

Multiple implementations can share a common interface. The Controller can depend on the interface rather than a concrete implementation. This provides loose coupling and makes it easier to replace or add implementations.

this vs target with JDK Dynamic Proxy

- this refers to the proxy, whose type is the interface.
- target refers to the actual target object, whose type is the concrete implementation.
- A Pointcut using this(StudentService.class) may not match a JDK proxy implementing only CommonService.
- The corresponding target(StudentService.class) can match the actual target.

With CGLIB, the proxy is a subclass of the target class, so the proxy can also match the target class type.

Key concept: this matches the proxy type; target matches the actual target type. The configured proxy mechanism affects the result.

17. Quick Comparison of Pointcut Designators

Designator	Matching purpose	Level
execution()	Method execution signature	Method
within()	Class or package	Class
@annotation	Annotation on a method	Method
bean()	Spring bean name	Bean
this	Proxy object's type	Proxy
target	Actual target object's type	Target
args	Runtime argument types	Method
@args	Annotations on argument types	Method
@within	Annotation on declaring type	Class
@target	Annotation on target object's type	Class

18. Interview Questions

1. What is a Pointcut in Spring AOP?
A Pointcut identifies the methods or classes where an Advice should be applied.

2. What are the optional parts of an execution() expression?
Access modifier and exception pattern are optional.

3. What is the difference between * and ..?
* matches one item in the relevant position; .. matches zero or more parameters or package levels in supported positions.

4. How can you intercept all methods inside a class?
Use within() with the class path, or execution() with a wildcard method pattern.

5. What is the difference between execution() and within()?
execution() matches method executions based on signatures; within() matches based on the class or package.

6. What is @annotation used for?
It matches methods carrying a specified annotation.

7. What is the purpose of bean()?
It matches methods belonging to a Spring bean identified by its bean name.

8. Which logical operators can combine Pointcuts?
AND (&&), OR (||), and NOT (!).

9. What is a named Pointcut?
A reusable Pointcut expression declared using @Pointcut and an empty method body.

10. Why create an ApplicationPointcuts class?
To centralize commonly used Pointcut definitions and reuse them across multiple Aspects.

11. What is the difference between this and target?
this matches the proxy object's type, whereas target matches the actual target object's type.

12. What is the difference between CGLIB and JDK Dynamic Proxy?
CGLIB creates a subclass-based proxy. JDK Dynamic Proxy creates a proxy implementing an interface.

19. Final Revision Flow

```
AOP
 |
 v
Pointcut
 |
 v
Pointcut Expression
 |
 v
Designator
 |
 v
execution()
 |
 v
Method Signature Matching
 |
 v
Wildcards
 |
 v
* and ..
 |
 v
within()
 |
 v
Class-Level Matching
 |
 v
@annotation
 |
 v
Method Annotation Matching
 |
 v
bean()
 |
 v
Bean Name Matching
 |
 v
this / target / args
 |
 v
Proxy and Argument Matching
 |
 v
Logical Operators
 |
 v
Named Pointcuts
 |
 v
ApplicationPointcuts Class
 |
 v
```

Reusable AOP Expressions

Most Important Things to Remember

- execution() is used for method-level matching.
- within() is used for class/package-level matching.
- * matches one item; .. matches zero or more items in supported positions.
- @annotation matches methods carrying a particular annotation.
- bean() matches by Spring bean name.
- this refers to the proxy; target refers to the actual target.
- @Pointcut allows expressions to be reused.
- Keep Pointcuts scoped to application packages to avoid unnecessarily intercepting framework internals.

End of Notes